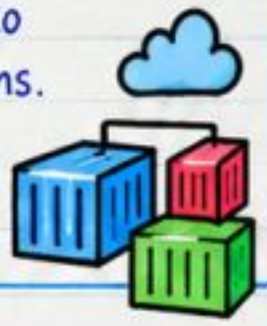
1. KUBERNETES

Kubernetes (K8s) is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications.

kubernetes

What is Kubernetes ?

- Kubernetes (K8s) is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications.
- It groups containers that make up an application into logical units for easy management and discovery.



Why Kubernetes ?

- To manage containers at scale.
- To automate deployment, scaling and operations.
- To ensure high availability and reliability.
- To optimize resource utilization.
- To provide portability across environments (cloud, on-prem, hybrid).

Key Features

1	Automation	- Automates deployment, scaling, and management.
2	Scalability	- Easily scale apps up or down based on demand.
3	Self-Healing	- Automatically restarts, replaces or reschedules failed containers.
4	High Availability	- Ensures minimal downtime with replication and failover.
5	Load Balancing	- Distributes traffic across containers.
6	Service Discovery	- Automatically discovers services and endpoints.
7	Rolling Updates & Rollbacks	- Update apps without downtime and rollback easily if needed.
8	Portability	- Run applications anywhere: on-prem, cloud or hybrid.

Common Use Cases

- Deploy and manage microservices
- Run stateless and stateful applications
- Handle traffic spikes with auto-scaling
- Manage multi-environment deployments

Benefits

- Faster and more reliable deployments
- Better resource utilization
- Improved application availability
- Consistent environments across dev, test and prod
- Supports cloud, on-prem and hybrid infrastructures

Rakesh Kumar

2. KUBERNETES ARCHITECTURE

A Kubernetes cluster consists of a Control Plane (Master Node) and Worker Nodes working together to run containerized applications.

1 Control Plane (Master Node)

- Brain of the cluster.
- Manages the cluster and makes global decisions.
- Handles scheduling, scaling, and maintaining desired state.

Control Plane Components

API Server
- Exposes the K8s API.
- Central point of communication.

etcd
- Consistent and highly available key-value store.

Scheduler
- Assigns Pods to nodes based on resources & rules.

Controller Manager
- Runs controllers to maintain the desired state.

Cloud Controller Manager
- Integrates with cloud providers to manage cloud resources.

2 Worker Node

- Runs the actual applications (containers).
- Executes pods as instructed by the control plane.
- Contains kubelet, kube-proxy and container runtime.

Worker Node Components

kubelet
- Ensures containers are running in a Pod.

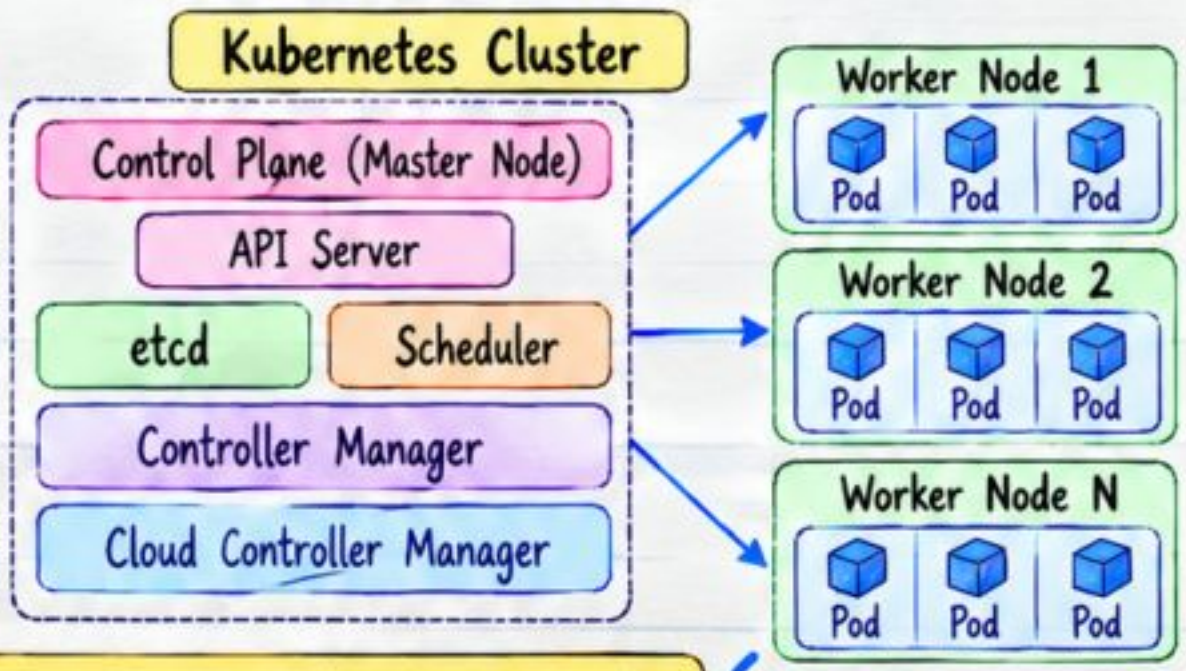
kube-proxy
- Manages network rules and service communication.

Container Runtime
- Runs containers (e.g., containerd, Docker).

Pods

3 Cluster Overview

- A Kubernetes cluster is a set of master (control plane) and worker nodes working together.
- Control plane manages the cluster.
- Worker nodes run the workloads (pods).



Control Plane thinks, Worker Nodes do the work.

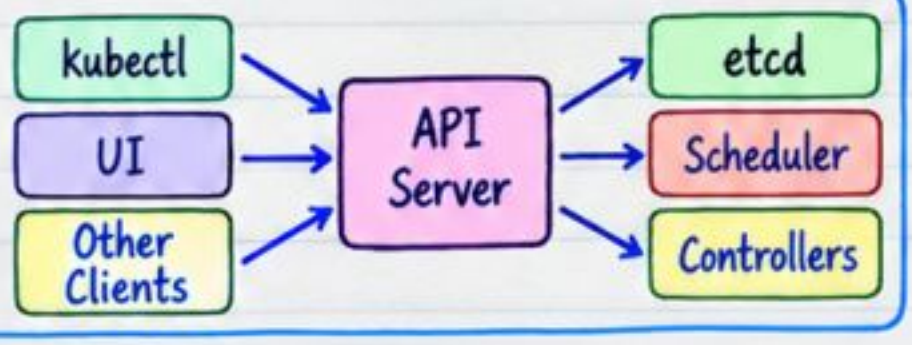
Rakesh Kumar

3. Core Components

These components are the main building blocks that make Kubernetes work.

1 API Server

- Frontend for the Kubernetes control plane.
- Exposes the Kubernetes API and is the central point of all communication.

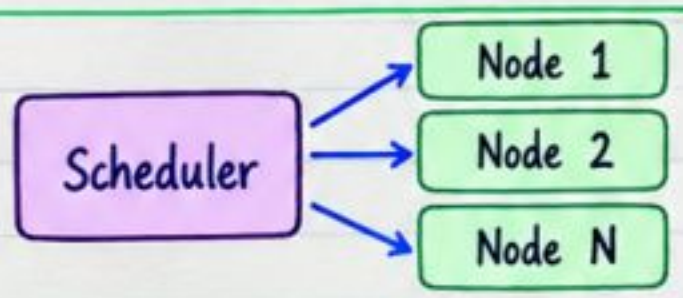


2 etcd

- Consistent and highly available key-value store.
- Stpres all cluster data and configuration in a reliable way.

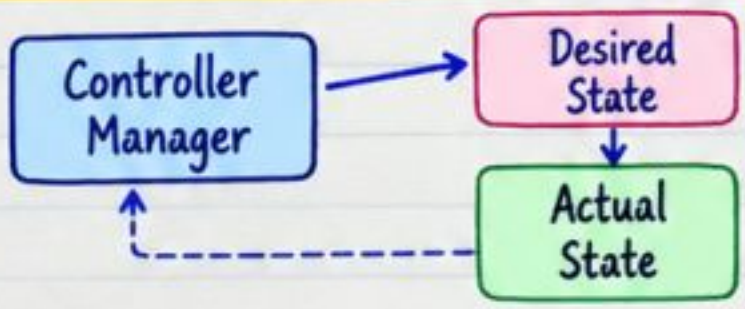
3 Scheduler

- Watches for newly created Pods with no node assigned.
- Selects the best node for the Pod to run on based on resources and policies.



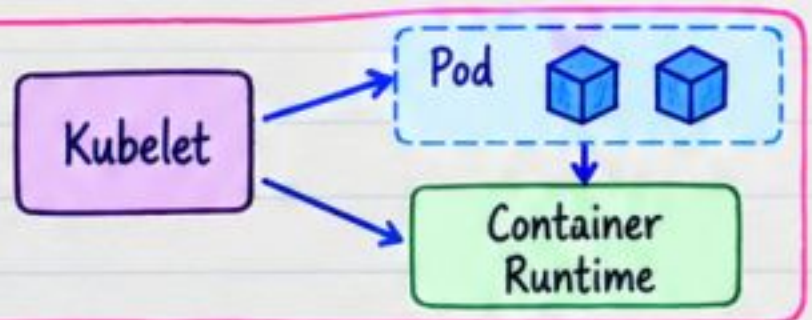
4 Controller Manager

- Runs controllers to maintain the desired state of the cluster.
- Examples: Node Controller, Replication Controller, Endpoint Controller, etc.



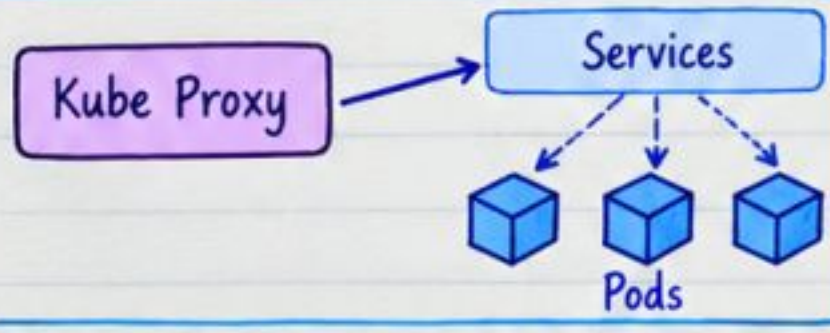
5 Kubelet

- Runs on each worker node.
- Ensures containers are running in a Pod.
- Communicates with the API Server.



6 Kube Proxy

- Runs on each node.
- Maintains network rules.
- Enables network communication to Pods from inside and outside the cluster.

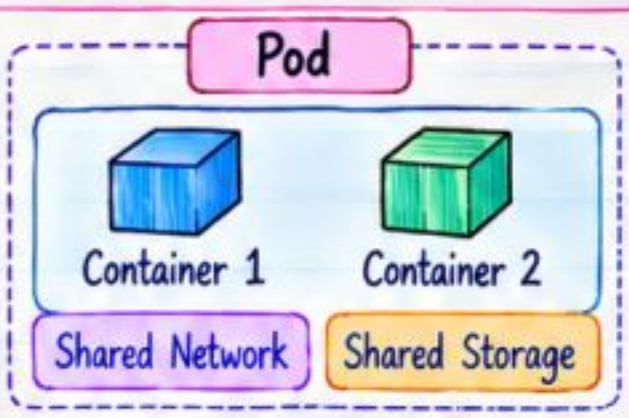


Together, these components ensure the cluster is healthy, scalable, and self-healing.

4. Pods, ReplicaSet, Deployment
Rolling Update & Rollback

1 Pods

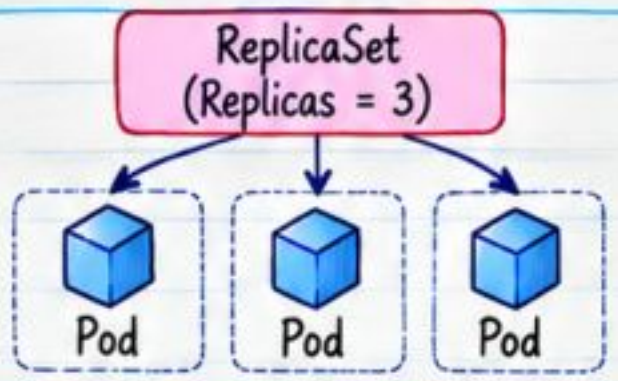
- The smallest deployable unit in Kubernetes.
- Represents one or more containers that share the same network, storage and configuration.
- Pods are ephemeral (can be created, destroyed and recreated).



Pods get their own IP and communicate with other Pods / Services.

2 ReplicaSet

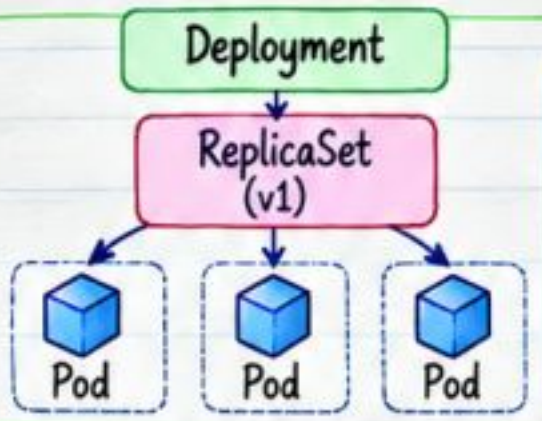
- Ensures that a specified number of Pod replicas are running at any given time.
- If a Pod fails, ReplicaSet creates a new Pod to maintain the desired count.



Maintains the desired number of identical Pods.

3 Deployment

- Manages ReplicaSets and provides declarative updates for Pods and ReplicaSets.
- Supports scaling, rolling updates and rollbacks.



Deployment manages ReplicaSets and ensures desired state of the application.

4 Rolling Update

- Updates Pods gradually with zero downtime.
- New Pods are created first, old Pods are terminated later.

Example (Replicas = 3)

5 Rollback

- Rolls back the Deployment to a previous version if something goes wrong.
- Restores the old ReplicaSet and Pods.

Quickly revert to a stable version.

Key Points

- Pod → smallest unit
- ReplicaSet → maintains desired pods
- Deployment → manages ReplicaSets, updates, scaling
- Rolling Update → zero downtime updates
- Rollback → revert to previous stable version

5. SERVICES

A Service in Kubernetes is a stable network endpoint that exposes a set of Pods as a network service. It provides load balancing, service discovery and a stable IP/DNS name.

Why Services ?

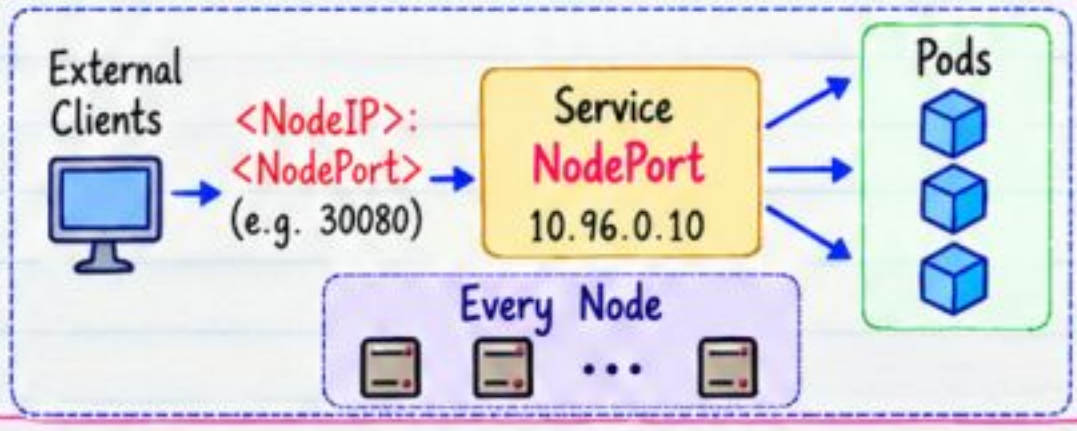
- Stable endpoint for Pods
- Load balancing
- Service discovery
- Decouples frontend & backend

1 ClusterIP (Default)

- Exposes the Service on a cluster-internal IP.
- Accessible only within the cluster.
- Default type if no type is specified.

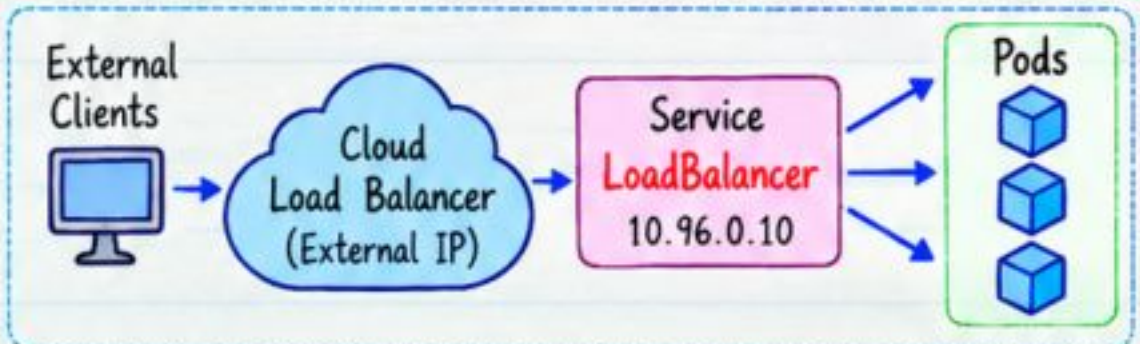
2 NodePort

- Exposes the Service on a static port on each Node's IP.
- Accessible from outside the cluster using <NodeIP>:<NodePort>.
- Port range: 30000 - 32767.



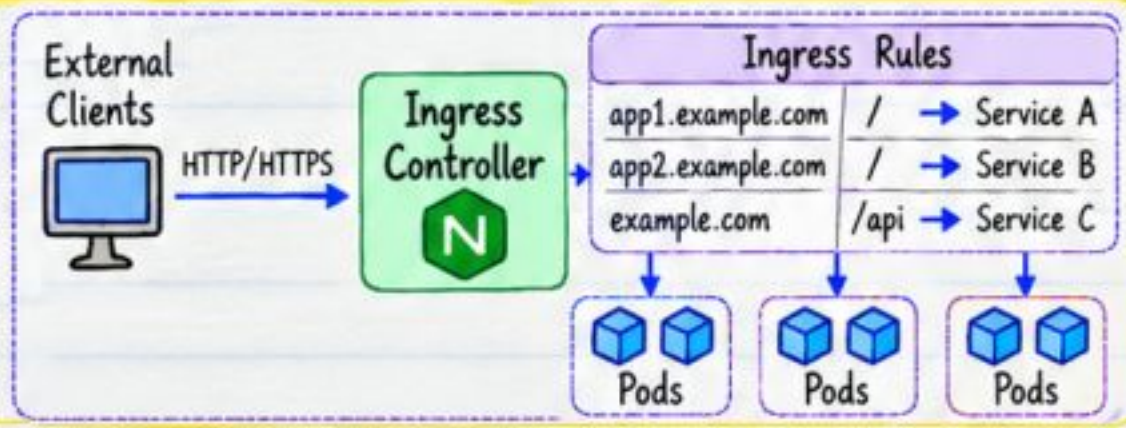
3 LoadBalancer

- Exposes the Service externally using a cloud provider's load balancer.
- Automatically provisions a load balancer (in supported clouds).
- Gets a stable external IP.



4 Ingress

- Provides HTTP/HTTPS routing to Services based on host names or paths.
- Uses an Ingress Controller (e.g. NGINX, Traefik) to manage the rules.
- Operates at Layer 7 (Application Layer).



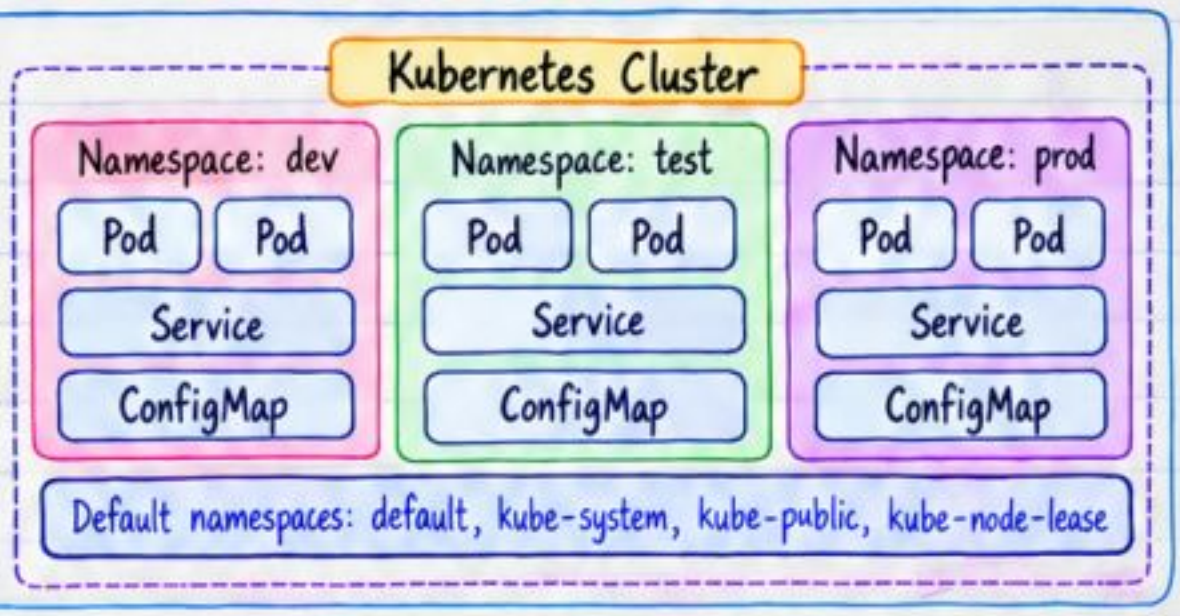
Quick Comparison

ClusterIP	NodePort	LoadBalancer	Ingress
Internal access only. Default.	External access via <NodeIP>:<NodePort>. Static port on every node.	External access via cloud load balancer. Auto-provisioned.	HTTP/HTTPS based routing. Path/host based rules. Needs Ingress Controller.

6. NAMESPACES, LABELS, SELECTORS, CONFIGMAP & SECRET

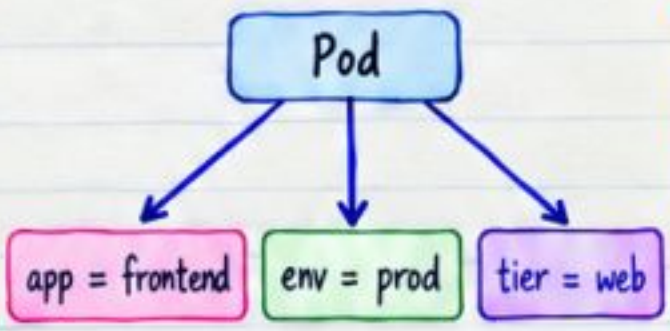
1 Namespaces

- Namespaces provide a way to divide cluster resources between multiple users or projects.
- Helps in organizing and isolating resources within the same cluster.
- Resources in one namespace cannot access resources in another namespace (by default).



2 Labels

- Labels are key-value pairs attached to Kubernetes objects.
- Used to organize and identify objects easily.
- Example: app=frontend, env=prod, tier=web

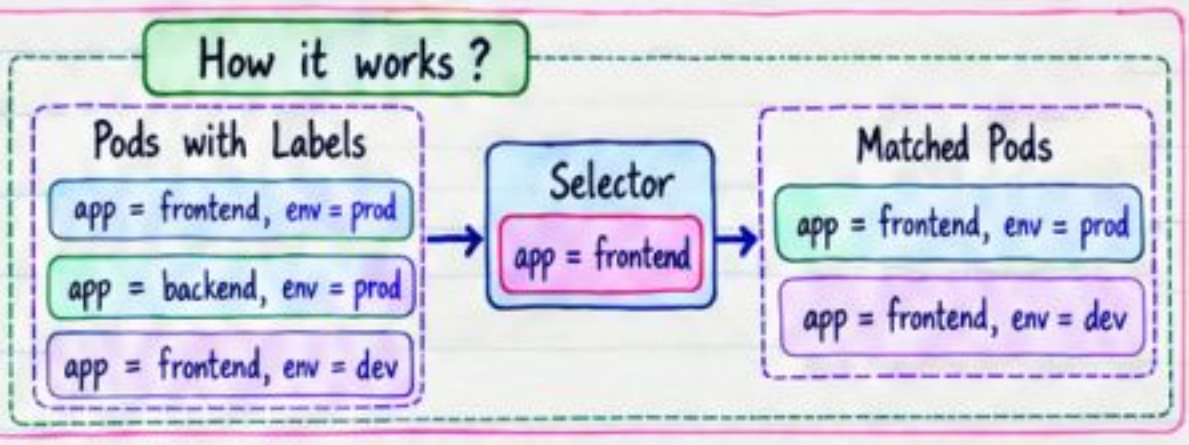


Common Labels (Examples)

```
app = frontend
env = prod
tier = web
version = v1
component = backend
```

3 Selectors

- Selectors are used to filter resources based on labels.
- Used by Services, ReplicaSets, Deployments to find matching Pods.
- Types: Equality-based, Set-based.

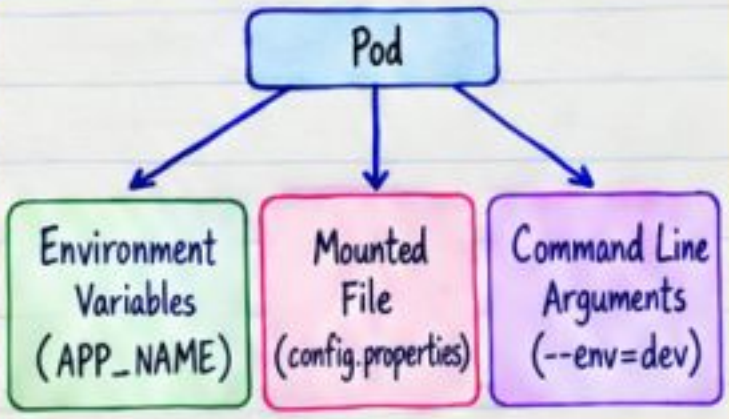


4 ConfigMap

- ConfigMap stores non-confidential configuration data in key-value pairs.
- Can be used as environment variables, command-line arguments or mounted files in a container.
- Helpful to separate config from code.

Example (YAML)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_NAME: my-app
  APP_ENV:
  LOG_LEVEL: info
```

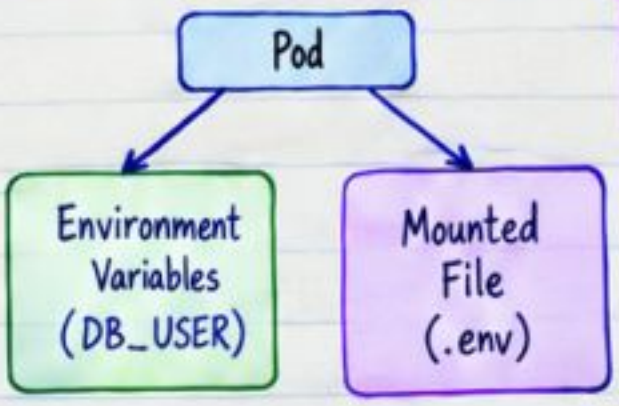


5 Secret

- Secret stores sensitive data like password, tokens, keys, etc.
- Data is stored in base64 encoded format (not encrypted by default).
- Used as environment variables or mounted files in a Pod.

Example (YAML)

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  DB_USER: cmk2bSh
  DB_PASSWORD: cGFsc33vcrd=
```



Key Takeaway

- Namespaces → divide and isolate resources
- Labels → key-value metadata
- Selectors → find and filter resources
- ConfigMap → non-confidential configuration
- Secret → sensitive data (base64 encoded)
- Used by Services, ReplicaSets, Deployments
- Helps in better organization and management

7. Volumes, PV, PVC & StorageClass

7

Kubernetes Volumes provide persistent storage for containers. PV, PVC and StorageClass work together to make storage dynamic, scalable and manageable.

Why Volumes ?

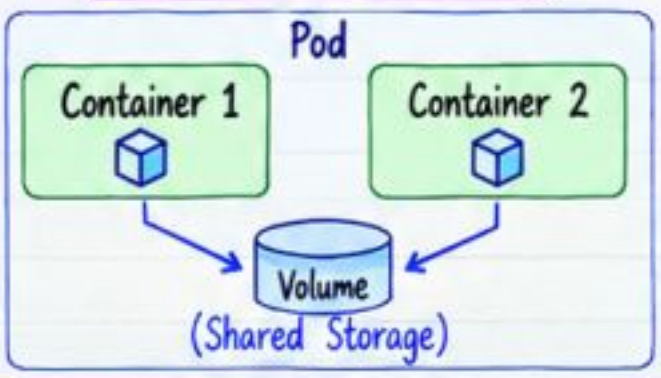
- Data persists beyond Pod lifecycle
- Required for stateful applications
- Share data across multiple Pods
- Different storage types (EBS, NFS, etc.)
- Enable dynamic and flexible storage

kubernetes

1 Volumes

- Volumes are directories, possibly with some data, which are mounted into containers in a Pod.
- Types: emptyDir, hostPath, configMap, secret, persistentVolume, etc.
- Provide a way to share data between containers in a Pod.

Pod with Volume



Volume Benefits

- Data available for the life of the Pod
- Shared between containers
- Manages persistent storage

2 Persistent Volume (PV)

- PV is a piece of storage in the cluster provisioned by an admin or dynamically using StorageClass.
- It is a cluster resource, independent of any Pod.
- PV can be created with various storage types (EBS, NFS, local, etc.).

Persistent Volume (PV)

- Provisioned by Admin or dynamically
- Exists in the cluster
- Not bound to any single Pod
- Stores data for long-term use
- Backend: EBS / NFS / local / other storage

PV Example (YAML)

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-demo
spec:
  capacity:
    storage: 5Gi
  accessModes: ["ReadWriteOnce"]
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /data/pv-demo
```

3 Persistent Volume Claim (PVC)

- PVC is a request for storage by a user.
- It is a namespaced resource.
- Binds to a suitable PV that matches its requirements.
- Used by Pods to access storage.

Persistent Volume Claim (PVC)

- Requested by user
- Namespaced resource
- Binds to a matching PV
- Specifies size & access modes
- Used in Pod to mount storage

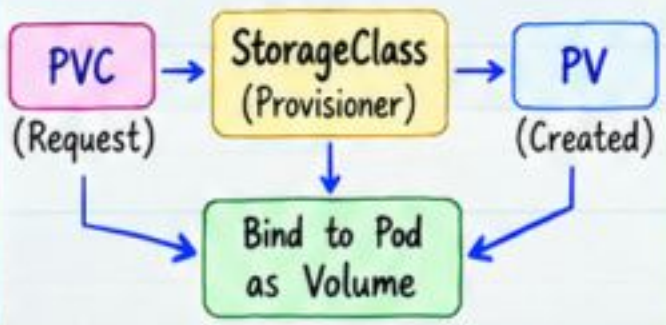
PVC Example (YAML)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 5Gi
```

4 StorageClass

- StorageClass provides a way to dynamically provision storage.
- Defines the type of storage (EBS, GP2, GP3, NFS, etc.).
- Helps in automatic provisioning of PV.
- PVC uses StorageClass to create PV dynamically.

StorageClass Flow



StorageClass Example (YAML)

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp2-storage
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
  fsType: ext4
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

Key Takeaways

- Volumes → share data in a Pod
- PV → cluster-level storage
- PVC → request storage (binds to PV)
- StorageClass → dynamic provisioning
- Data persists beyond Pod lifecycle
- Used for stateful applications
- Supports multiple storage types (EBS, NFS, etc.)

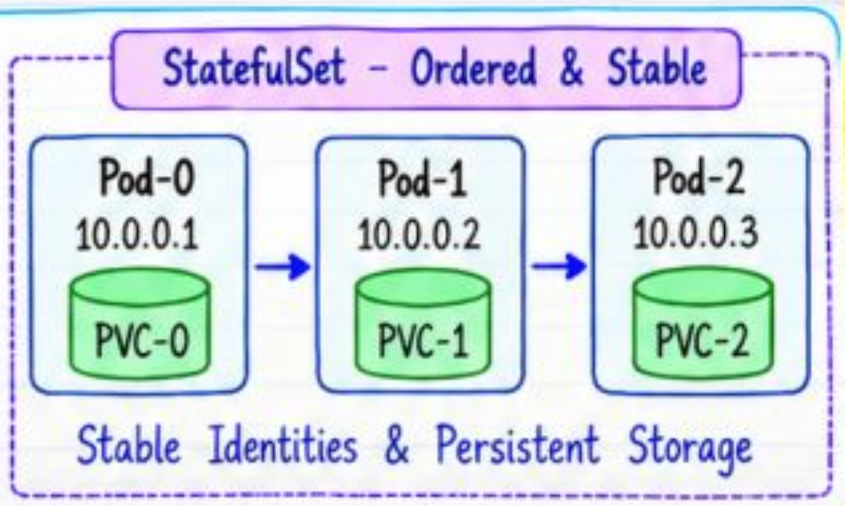
Rakesh Kumar

8. StatefulSet, DaemonSet, Job & CronJob

8

1 StatefulSet

- Manages stateful applications.
- Provides stable, unique network identities and persistent storage.
- Pods are created in order and deleted in reverse order.
- Examples: Databases (MySQL, MongoDB), Kafka, etc.

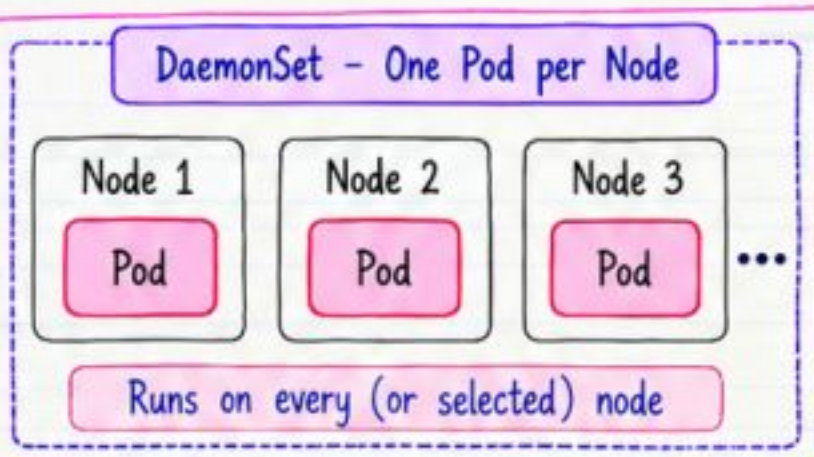


Example (YAML)

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels: { app: mysql }
```

2 DaemonSet

- Ensures that a copy of a Pod runs on all (or selected) Nodes.
- Useful for node-level tasks like logging, monitoring, networking.
- When a new node is added, a Pod is automatically scheduled on it.

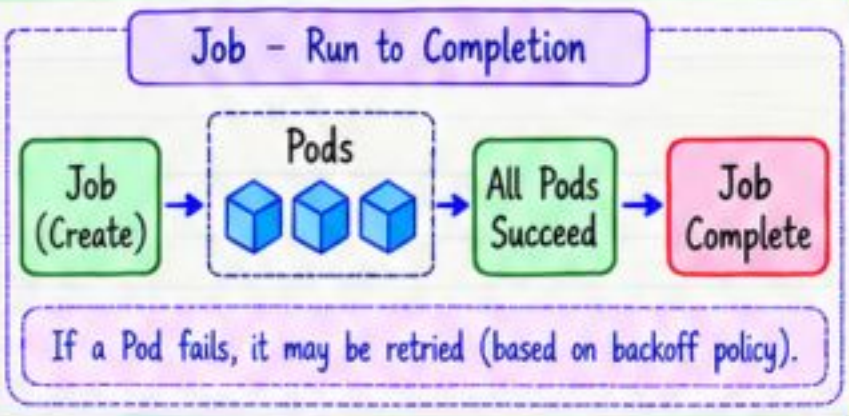


Example (YAML)

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
spec:
  selector:
    matchLabels:
      app: fluentd
  template:
    metadate:
      labels: { app: fluentd }
```

3 Job

- Creates Pods that run to completion and then stop.
- Suitable for one-time tasks like batch processing, data migration.
- Ensures that a specified number of Pods successfully complete.

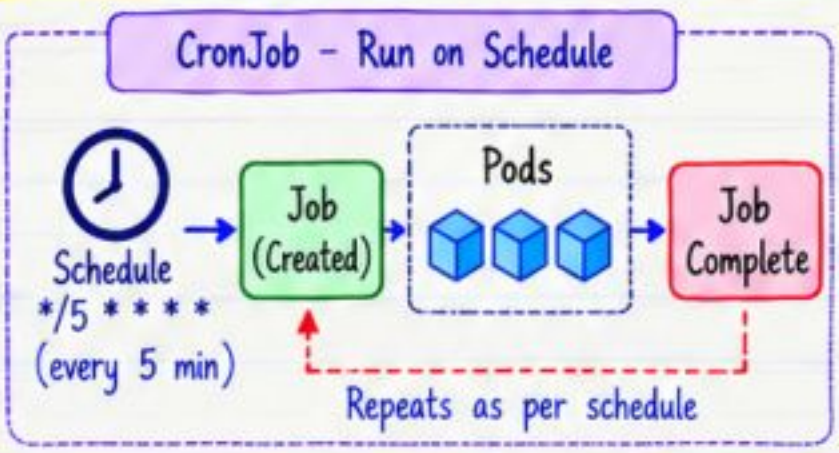


Example (YAML)

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: data-migration
spec:
  completions: 1
  template:
    spec:
      containers:
      - name: migrator
        image: my-app
      restartPolicy: OnFailure
```

4 CronJob

- Runs Jobs on a schedule (like cron).
- Useful for recurring tasks such as backups, reports, cleanup jobs.
- Creates a Job at the scheduled time.
- Example schedule format: "*/5 * * * *" (every 5 minutes).



Example (YAML)

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-job
spec:
  schedule: "*/5 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: my-backup
          restartPolicy: OnFailure
```

Quick Comparison

Workload	Purpose	Key Points	Example Use Cases
StatefulSet	Stateful applications	Stable identity, ordered, persistent storage	Databases, Kafka, etc.
DaemonSet	Run on all nodes	One Pod per node	Logging agents, monitoring, CNI
Job	Run once to completion	Terminates after success/failure	Batch jobs, data migration
CronJob	Run on a schedule	Creates Jobs based on cron schedule	Backups, reports, cleanup

Think: Deployment = stateless | StatefulSet = stateful | Job/CronJob = tasks | DaemonSet = nodes

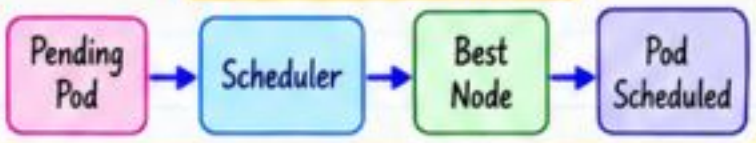
Rakesh Kumar

9. SCHEDULING

9

Scheduling is the process of assigning Pods to Nodes based on resource requirements, policies and constraints.

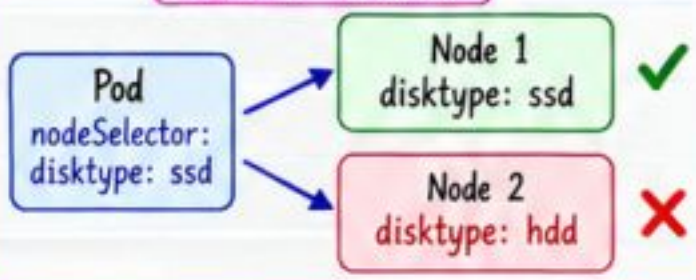
Scheduler Flow



1 Node Selector

- Assigns Pods to Nodes with specific labels.
- Simple way to control placement.
- Uses exact match.

How it works ?



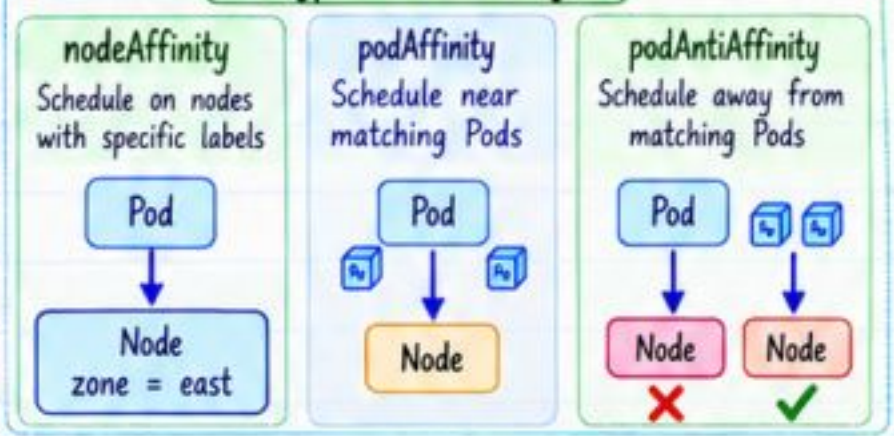
Example (YAML)

```yaml
spec:
  nodeSelector:
    disktype: ssd
    env: prod
```

2 Affinity

- More advanced rules for scheduling.
- Types:
  1. nodeAffinity (on nodes)
  2. podAffinity (with other Pods)
  3. podAntiAffinity (avoid Pods)
- Supports soft (preferred) & hard (required) rules.

Types of Affinity



Example (YAML)

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disktype
          operator: In
          values: [ssd]
```

3 Taints

- Applied to Nodes to repel Pods.
- Pods without matching toleration will not be scheduled.
- Helps reserve Nodes for specific workloads.

How it works ?

Example (YAML)

```yaml
# Taint on Node
kubectl taint nodes node1 dedicated=api:NoSchedule
# Pod with toleration
tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "api"
    effect: "NoSchedule"
```

4 Tolerations

- Applied to Pods to allow them to be scheduled on tainted Nodes.
- Must match the taint key, value and effect.

How it works ?

Example (YAML)

```yaml
tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "api"
    effect: "NoSchedule"
```

5 Taints & Tolerations Together

- Taints repel Pods.
- Tolerations allow Pods.
- Used together to control where Pods can run.

Summary

Common Effects

- NoSchedule → Pod won't be scheduled
- PreferNoSchedule → Try to avoid Node
- NoExecute → Evict existing Pods (if toleration not present)

Quick Comparison

Concept	Purpose	Applies To	Effect / Use Case
Node Selector	Simple label based selection	Pod	Schedules Pod on matching labeled Nodes
Affinity	Advanced placement rules	Pod	Place Pods near/away from other Pods
Taints	Repel Pods from Nodes	Node	Pods without toleration won't be scheduled
Tolerations	Allow Pods on tainted Nodes	Pod	Must match taint key, value and effect

Think: Taints control which Pods can run on a Node || Tolerations allow Pods to run on tainted Nodes.

10. SECURITY

10

Kubernetes provides multiple security mechanisms to protect your cluster, applications and data.

Security Goals

- Authentication – Who are you?
- Authorization – What can you do?
- Isolation – Keep workloads separated
- Audit & Monitoring – Track & detect issues

1 RBAC (Role Based Access Control)

- Controls who can perform actions on Kubernetes resources.
- Uses Roles (namespace scoped) and ClusterRoles (cluster scoped).
- Bind them to users, groups or service accounts.
- Follows the principle of least privilege.

How it works ?

Example Flow

Example (YAML)

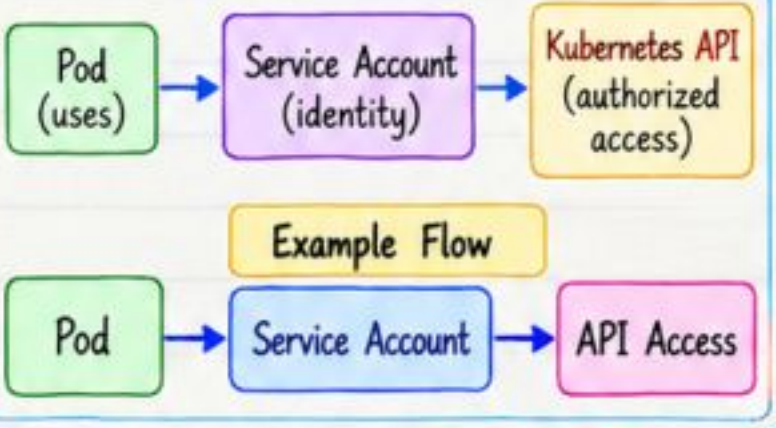
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]

kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
  - kind: User
    name: alice
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

2 Service Account

- Provides an identity for processes running in Pods.
- Used by Pods to interact with the Kubernetes API.
- Each namespace has a default Service Account.
- Use least privilege permissions.

How it works ?



Example Flow

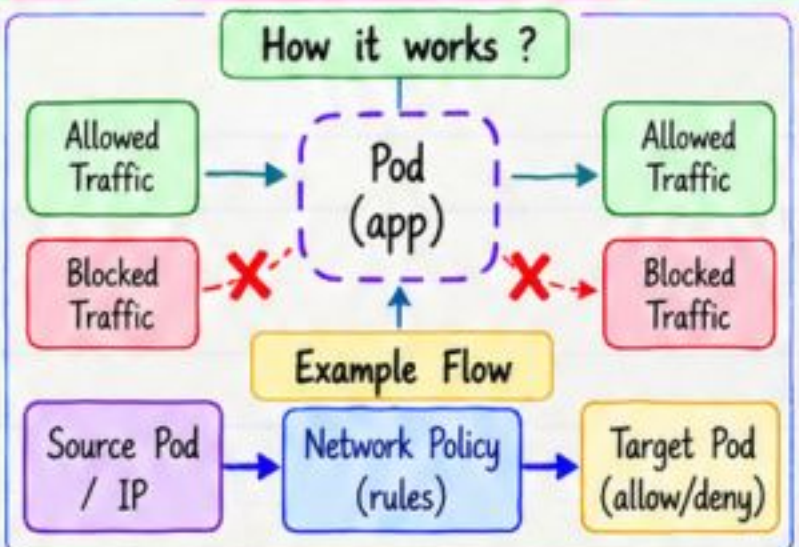
Example (YAML)

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-sa
  namespace: dev
---
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: dev
spec:
  serviceAccountName: my-sa
  containers:
    - name: app
      image: nginx
```

3 Network Policy

- Controls the traffic flow to and from Pods.
- Define allowed/denied rules based on Pods, namespaces, IP blocks and ports.
- Provides isolation between applications.

How it works ?



Example Flow

Example (YAML)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: app
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 80
```

Key Takeaways

- RBAC controls what you can do.
- Service Account is the identity for applications.
- Network Policies control how Pods communicate.
- Follow least privilege and defense in depth.

Best Practices

- ✔ Use RBAC – don't use cluster-admin.
- ✔ Create dedicated Service Accounts.
- ✔ Restrict network traffic with Network Policies.
- ✔ Keep monitoring & auditing enabled.

Rakesh Kumar

11. KUBECTL CHEAT SHEET

Top 30-40 Most Used Commands

kubernetes

Tip:

- Use kubectl get <resource> -o wide for more details.
- Add -n <namespace> to run in a specific namespace.

Abbreviations

ns	= namespace	pv	= persistent volume
svc	= service	pvc	= persistent volume claim
rs	= replicaset	cm	= configmap
po	= pod	secret	= secret
deploy	= deployment	ing	= ingress

#	Command	Purpose
1	kubectl version	Show client & server version
2	kubectl cluster-info	Display cluster information
3	kubectl get nodes	List all nodes
4	kubectl get nodes -o wide	List nodes with details
5	kubectl get pods	List pods in current ns
6	kubectl get pods -A	List pods in all namespaces
7	kubectl get pods -o wide	List pods with node/IP info
8	kubectl describe pod <pod>	Show details of a pod
9	kubectl logs <pod>	Show logs of a pod
10	kubectl logs -f <pod>	Stream logs of a pod
11	kubectl exec -it <pod> --/bin/sh	Open shell in a pod
12	kubectl get all	List all resources in ns
13	kubectl get all -A	List all resources in all ns
14	kubectl get svc	List services
15	kubectl get svc -o wide	List services with details
16	kubectl describe svc <svc>	Show details of a service
17	kubectl get deploy	List deployments
18	kubectl get deploy -o wide	List deployments with details
19	kubectl describe deploy <deploy>	Show deployment details
20	kubectl rollout status deploy/<deploy>	Check rollout status

#	Command	Purpose
21	kubectl rollout history deploy/<deploy>	Show rollout history
22	kubectl rollout undo deploy/<deploy>	Rollback to previous version
23	kubectl get rs	List ReplicaSets
24	kubectl get rs -o wide	List ReplicaSets with details
25	kubectl get ns	List namespaces
26	kubectl get cm	List ConfigMaps
27	kubectl describe cm <cm>	Show ConfigMap details
28	kubectl get secret	List Secrets
29	kubectl describe secret <secret>	Show Secret details
30	kubectl get pv	List Persistent Volumes
31	kubectl get pvc	List Persistent Volume Claims
32	kubectl describe pvc <pvc>	Show PVC details
33	kubectl get ingress	List Ingress resources
34	kubectl describe ingress <ing>	Show Ingress details
35	kubectl top nodes	Show node resource usage
36	kubectl top pods	Show pod resource usage
37	kubectl apply -f <file.yaml>	Create/Update from YAML
38	kubectl delete -f <file.yaml>	Delete resources from YAML
39	kubectl delete pod <pod>	Delete a pod
40	kubectl drain <node> --ignore-daemonsets	Drain a node for maintenance

Useful Flags

- -n <namespace> Run command in a namespace
- -A Run command in all namespaces
- -o wide Show more details
- -o yaml Output in YAML format
- -o json Output in JSON format
- -w Watch for changes
- -l <label> Filter by label
- --all-namespaces Same as -A

Common Resource Types

- pod - Pod
- svc - Service
- deploy - Deployment
- rs - ReplicaSet
- ns - Namespace
- pv - PersistentVolume
- pvc - PersistentVolumeClaim
- cm - ConfigMap
- secret - Secret
- ing - Ingress
- job - Job
- cronjob - CronJob
- ds - DaemonSet
- sts - StatefulSet

Remember

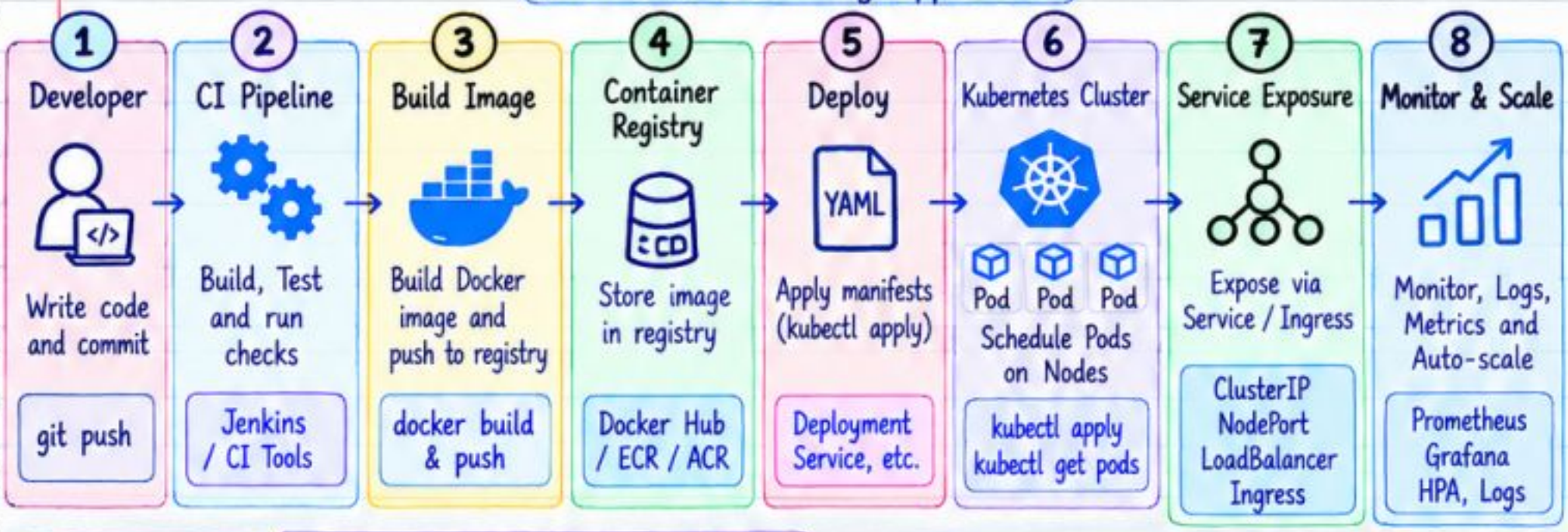
- Everything in Kubernetes is API driven.
- Most kubectl commands follow the pattern: kubectl <command> <resource>.
- Use describe & logs for troubleshooting.
- Use apply for idempotent changes.
- Always use proper namespaces and labels.

Rakesh Kumar

12. Kubernetes Workflow

End-to-End Deployment Flow

From Code to Running Application



Kubernetes Workflow Details

#	Step	Description	Tools / Commands
1	Developer	Write code and commit to git repository.	git, GitHub
2	CI Pipeline	Trigger build, run tests, code scan, quality checks.	Jenkins, GitLab CI, GitHub Actions
3	Build Image	Build Docker image and push to registry.	docker build, docker push
4	Container Registry	Store and manage container images securely.	Docker Hub, AWS ECR, Azure ACR, GCR
5	Deploy	Apply Kubernetes manifests to create resources.	kubectl apply -f
6	Kubernetes Cluster	Scheduler places Pods on available Nodes and maintains desired state.	kubectl get pods, kubectl get nodes
7	Service Exposure	Expose application using Service/Ingress.	ClusterIP, NodePort, LoadBalancer, Ingress
8	Monitor & Scale	Monitor health, collect metrics and auto-scale the application.	Prometheus, Grafana, HPA, Logs (ELK/Splunk)

Common Resources in Flow

- **Deployment / StatefulSet** – Manage stateless or stateful applications.
- **Pod** – Smallest deployable unit.
- **Service** – Stable networking to access Pods.
- **ConfigMap / Secret** – Non-sensitive and sensitive data.
- **PersistentVolumeClaim** – Claim storage.
- **Ingress** – HTTP/HTTPS routing.
- **NetworkPolicy** – Control traffic.
- **ServiceAccount & RBAC** – Identity and access control.

Interview Revision Cheat Sheet

1. Basics
- What is Kubernetes?
- Why Kubernetes?
- K8s Architecture?
- Master & Worker Nodes?
- What is a Pod?

2. Core Concepts
- Deployment vs ReplicaSet
- Service – types and purpose
- ConfigMap vs Secret
- Namespace – logical isolation

3. Storage
- PV – cluster storage resource
- PVC – claim storage
- StorageClass – dynamic provisioning

4. Workloads
- StatefulSet – for stateful apps
- DaemonSet – runs on all nodes
- Job – run a task till completion
- CronJob – run job on schedule

5. Networking
- Pod IP – ephemeral
- Service – ClusterIP, NodePort, LoadBalancer
- Ingress – HTTP/HTTPS routing

6. Scheduling
- Node Selector – schedule on specific nodes
- Affinity – advanced rules
- Taints & Tolerations – allow/repel pods

7. Security
- RBAC – role based access control
- ServiceAccount – identity for Pods
- NetworkPolicy – control traffic

8. Observability
- kubectl logs – view logs
- kubectl describe – details
- kubectl top – resource usage
- Events – troubleshooting
- Prometheus + Grafana – monitoring

9. kubectl Essentials
- get – list resources
- describe – details
- apply – create/update
- delete – remove
- logs – pod logs
- exec – run command in pod
- scale – scale resources
- rollout – rollout updates
- port-forward – local access
- top – resource usage

10. Common YAMLs
- Deployment.yaml
- Service.yaml
- ConfigMap.yaml
- Secret.yaml
- Ingress.yaml

11. Best Practices
- Use namespaces
- Use labels & selectors
- Keep containers lightweight
- Use ConfigMaps & Secrets
- Set resource requests/limits

Remember

- Understand concepts before memorizing commands.
- Practice YAML manifest writing regularly.
- Use kubectl describe for troubleshooting.
- Observe, monitor and improve continuously.

Rakesh Kumar